

World Happiness Report — Nettoyage et préparation des données

À propos du projet

Le World Happiness Report est un rapport annuel publié par les Nations Unies qui mesure et classe le bonheur des populations à l'échelle mondiale. Chaque pays reçoit un score calculé à partir de six facteurs principaux : le PIB par habitant, le soutien social, l'espérance de vie, la liberté individuelle, la générosité et la perception de la corruption.

Ce notebook couvre les années 2015 à 2019. Le problème de départ était simple : chaque année correspond à un fichier CSV distinct, et ces fichiers ne partagent pas la même structure. Certains ont des colonnes absentes, d'autres utilisent des noms différents pour les mêmes variables. L'objectif était de nettoyer et fusionner ces 5 fichiers en un seul dataset cohérent, prêt pour l'analyse.

Fichiers sources

Fichier	Lignes	Particularités
2015.csv	158	Fichier de référence — colonnes <code>Region</code> et <code>Standard Error</code> présentes
2016.csv	157	Pas de <code>Standard Error</code> , remplacé par <code>Lower/Upper Confidence Interval</code>
2017.csv	155	Pas de colonne <code>Region</code> , noms de colonnes avec des points (<code>Happiness.Score</code>)
2018.csv	156	Pas de <code>Region</code> , colonnes renommées (<code>Score</code> , <code>GDP per capita</code> , <code>Social support</code>)
2019.csv	156	Même structure que 2018, 1 valeur manquante sur <code>Perceptions of corruption</code>

Description des colonnes du dataset final

Colonne	Type	Description
Country	str	Nom du pays
Region	str	Région géographique (10 régions + <code>Unknown</code>)
Happiness Rank	int	Classement du pays (1 = plus heureux)
Happiness Score	float	Score global de bonheur (0 à 10)
Standard Error	float	Erreur standard associée au score
Economy (GDP per Capita)	float	Contribution du PIB par habitant au score
Family	float	Contribution du soutien social et familial
Health (Life Expectancy)	float	Contribution de l'espérance de vie en bonne santé
Freedom	float	Contribution de la liberté de choix de vie
Trust (Government Corruption)	float	Contribution de la confiance envers le gouvernement
Generosity	float	Contribution de la générosité
Dystopia Residual	float	Résidu inexpliqué par les 6 facteurs précédents
Year	int	Année de l'observation (2015 à 2019)

Étapes du nettoyage

1. Exploration individuelle de chaque fichier

Avant toute fusion, j'ai examiné chaque fichier séparément pour identifier les différences de structure, les valeurs manquantes et les types de données. Les trois commandes utilisées systématiquement :

```
df.info()
df.isnull().sum()
df.describe().transpose()
```

C'est une étape que l'on est tenté de sauter pour aller plus vite, mais fusionner des fichiers sans les avoir explorés revient à importer des problèmes sans le savoir.

2. Reconstruction de `Standard Error` pour 2016 et 2017

Les fichiers 2016 et 2017 ne contiennent pas directement le `Standard Error`. Pour 2016, les colonnes `Lower Confidence Interval` et `Upper Confidence Interval` sont disponibles. Pour 2017, les colonnes `Whisker.low` et `Whisker.high` jouent le même rôle. J'ai utilisé la formule basée sur le z-score d'un intervalle de confiance à 90% ($z = 1.645$) :

```
df_2016['Standard Error'] = (
    df_2016['Upper Confidence Interval'] - df_2016['Lower Confidence Interval']
) / 2 * 1.645
```

Plutôt que de laisser cette colonne vide ou de la remplir par la médiane, cette approche reconstruit l'information depuis ce qui était disponible dans les données d'origine.

3. Harmonisation des noms de colonnes

Les fichiers 2017, 2018 et 2019 utilisent des conventions de nommage différentes. J'ai réordonné et renommé les colonnes pour qu'elles correspondent exactement à la structure du fichier 2015, utilisé comme référence :

```
df_2017.columns = df_2015.columns
df_2018_2019.columns = df_2015_a_2017.columns
```

4. Ajout de la colonne `Year`

J'ai ajouté une colonne `Year` à chaque fichier avant la fusion pour conserver l'origine temporelle de chaque ligne.

5. Fusion progressive

La fusion a été réalisée en deux temps pour gérer les différences de structure :

- `df_2015 + df_2016 + df_2017 → df_2015_a_2017`
- `df_2018 + df_2019 → df_2018_2019`
- `df_2015_a_2017 + df_2018_2019 → df` (dataset final)

6. Gestion des valeurs manquantes

Colonne	Méthode	Justification
Region	<code>groupby('Country').transform('first')</code>	Propagation depuis les années où la région est connue
Region (restant)	<code>fillna('Unknown')</code>	Pays absents de 2015–2016, région impossible à déterminer
Standard Error	Médiane globale	Colonne absente pour 2018–2019
Dystopia Residual	Médiane globale	Colonne absente pour 2018–2019
Perceptions of corruption	Médiane globale	1 valeur manquante dans le fichier 2018

Explications de quelques lignes de code

Récupérer la colonne `Region` manquante

```
df['Region'] = df['Region'].fillna(  
    df.groupby('Country')['Region'].transform('first')  
)
```

Les fichiers 2017, 2018 et 2019 ne contiennent pas de colonne `Region`. Mais en réfléchissant, je me suis dit que ces mêmes pays apparaissent déjà dans les fichiers 2015 et 2016 où la région est renseignée. Donc l'information existe — elle est juste dans d'autres lignes du dataset fusionné.

L'idée était la suivante : si la France est classée en "Western Europe" en 2015, elle l'est forcément aussi en 2017 et 2019. Pourquoi créer une valeur manquante quand on peut simplement la propager ?

`groupby('Country')` regroupe toutes les lignes d'un même pays ensemble. `transform('first')` va chercher la première valeur non-nulle de `Region` dans ce groupe et la répercute sur toutes les lignes du groupe — y compris celles qui n'avaient pas de région. Et `fillna()` ne remplace que les cases vides, sans écraser ce qui est déjà là.

Vérifier les valeurs manquantes après chaque étape

```
100 * df.isnull().sum() / len(df)
```

J'ai relancé cette ligne après chaque transformation, pas seulement à la fin. L'idée était de m'assurer à chaque étape que le nettoyage avançait dans le bon sens, plutôt que de découvrir un problème une fois toutes les opérations enchaînées.

`isnull()` crée un dataframe de booléens — True là où il y a un NaN, False ailleurs. `sum()` additionne les True par colonne pour obtenir le nombre de valeurs manquantes. En divisant par `len(df)` et en multipliant par 100, on obtient un pourcentage directement lisible.

Reconstruire Standard Error depuis les intervalles de confiance

```
df_2016['Standard Error'] = (
    df_2016['Upper Confidence Interval'] - df_2016['Lower Confidence Interval']
) / 2 * 1.645
```

Le fichier 2016 ne contient pas de `Standard Error` directement, mais il fournit les bornes inférieure et supérieure de l'intervalle de confiance à 90%. J'ai utilisé ça plutôt que de simplement remplir la colonne avec la médiane.

La logique derrière : un intervalle de confiance à 90% se construit selon la formule $[\text{estimation} - z * SE, \text{estimation} + z * SE]$, où z vaut 1.645 pour 90%. La largeur totale de l'intervalle est donc $2 * z * SE$. En isolant SE , on obtient $SE = (\text{Upper} - \text{Lower}) / 2 * 1.645$. C'est une reconstruction approximative, mais elle est ancrée dans la réalité des données plutôt qu'inventée.

Dataset final

```
Dimensions      : 776 lignes x 13 colonnes
Période         : 2015 - 2019
Pays distincts  : 168
Régions         : 10 régions géographiques + Unknown
Valeurs nulles  : 0
```

Prérequis

```
pip install pandas numpy
```

Utilisation

1. Cloner le dépôt et placer les 5 fichiers CSV dans le même dossier que le notebook
2. Lancer `World_Happiness_Report.ipynb` cellule par cellule
3. Le fichier `World_happiness_clean.csv` est généré automatiquement à la dernière cellule

```
World_Happiness_Report/
|
├─ World_Happiness_Report.ipynb  <- notebook principal
├─ World_happiness_clean.csv      <- dataset final (généré)
```

- └─ 2015.csv
- └─ 2016.csv
- └─ 2017.csv
- └─ 2018.csv
- └─ 2019.csv

Source des données

World Happiness Report — disponible sur [Kaggle](#).

Auteur

Isaac Houndjo